Travaux Pratiques 1 : Initiation à Docker

Institut Supérieur d'Informatique

Département Génie des Télécommunications et Réseaux (GTR)

Module : Cloud Computing & Virtualisation

Groupes : M1 SSII

Enseignant : Safa Réjichi

Réalisé par : Talel Chaanbi

Objectifs

- Préparation et installation de Docker sur une MV « Ubuntu ».
- Manipuler des images Docker.
- Manipuler des conteneurs Docker.

Partie 1 : Préparation de Docker et configurer le dépôt

I. Création d'une machine virtuelle avec VirtualBox

1. Téléchargement de VirtualBox

Rendez-vous sur le site de Oracle Virtual Box afin de télécharger la dernière version du logiciel :
https://www.virtualbox.org/wiki/Downloads

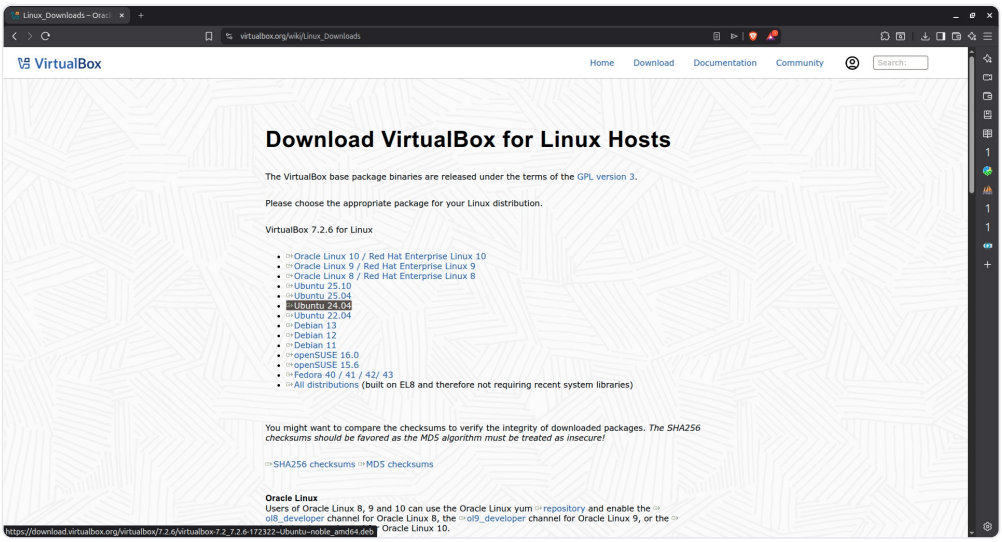


Figure 1 : Page officielle de téléchargement de VirtualBox

---

2. Téléchargement d'Ubuntu

Rendez-vous sur le site de Ubuntu afin de télécharger la dernière version de l'OS :
https://ubuntu.com/download/desktop

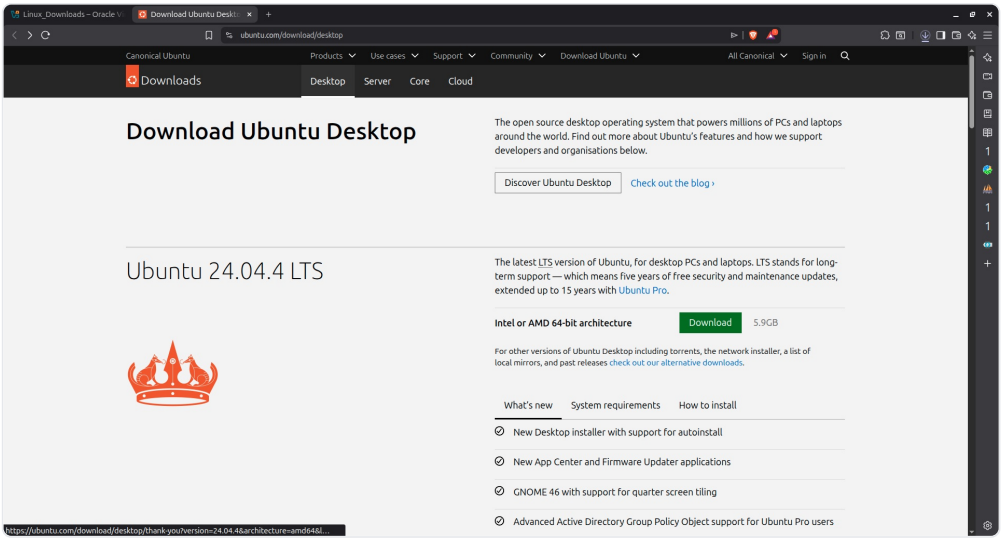


Figure 2 : Page officielle de téléchargement d'Ubuntu Desktop

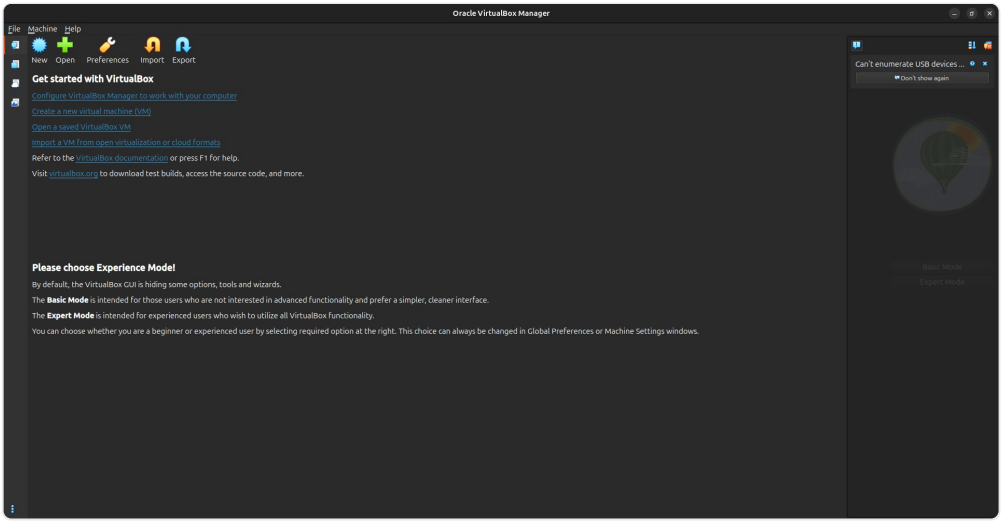


Figure 3 : Interface principale du logiciel VirtualBox

3. Création de la machine virtuelle

Créez la machine virtuelle appelée **Ubuntu** avec les paramètres suivants :

Paramètre	Valeur
Nom de la machine	Ubuntu
Mémoire RAM	3072 Mo
Disque dur	30 Go

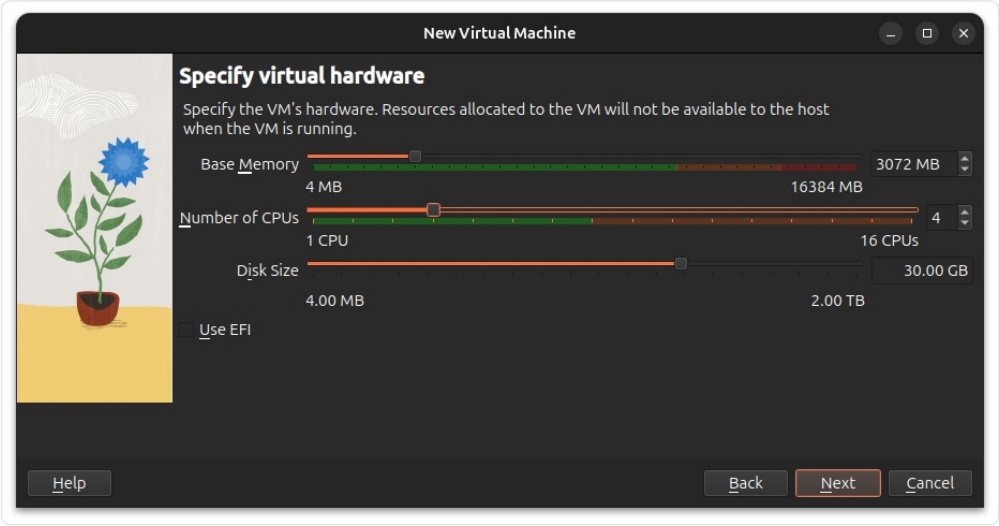


Figure 4 : Configuration de la machine virtuelle Ubuntu (RAM : 3072 Mo, Disque : 30 Go)

II. Installation de Docker

Il existe deux versions de Docker pour les systèmes Linux : **Docker CE** (Community Edition) et **Docker EE** (Enterprise Edition). La version CE est gratuite et open-source.

1. Mise à jour de la liste des packages

```
sudo apt-get update
```

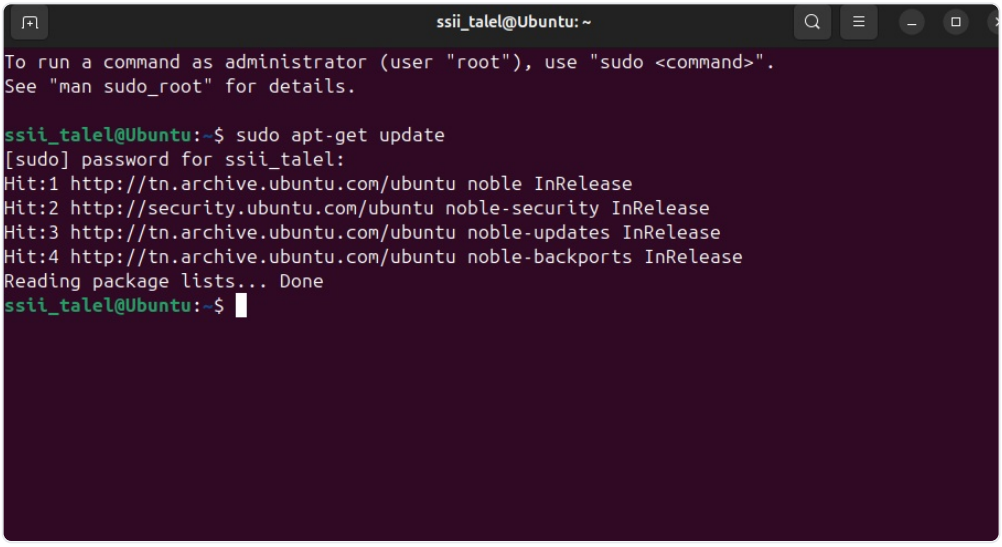


Figure 5 : Mise à jour de la liste des packages (`sudo apt-get update`)

---

2. Installation des paquets pré-requis

```
sudo apt-get install apt-transport-https ca-certificates curl software-properties-common
```

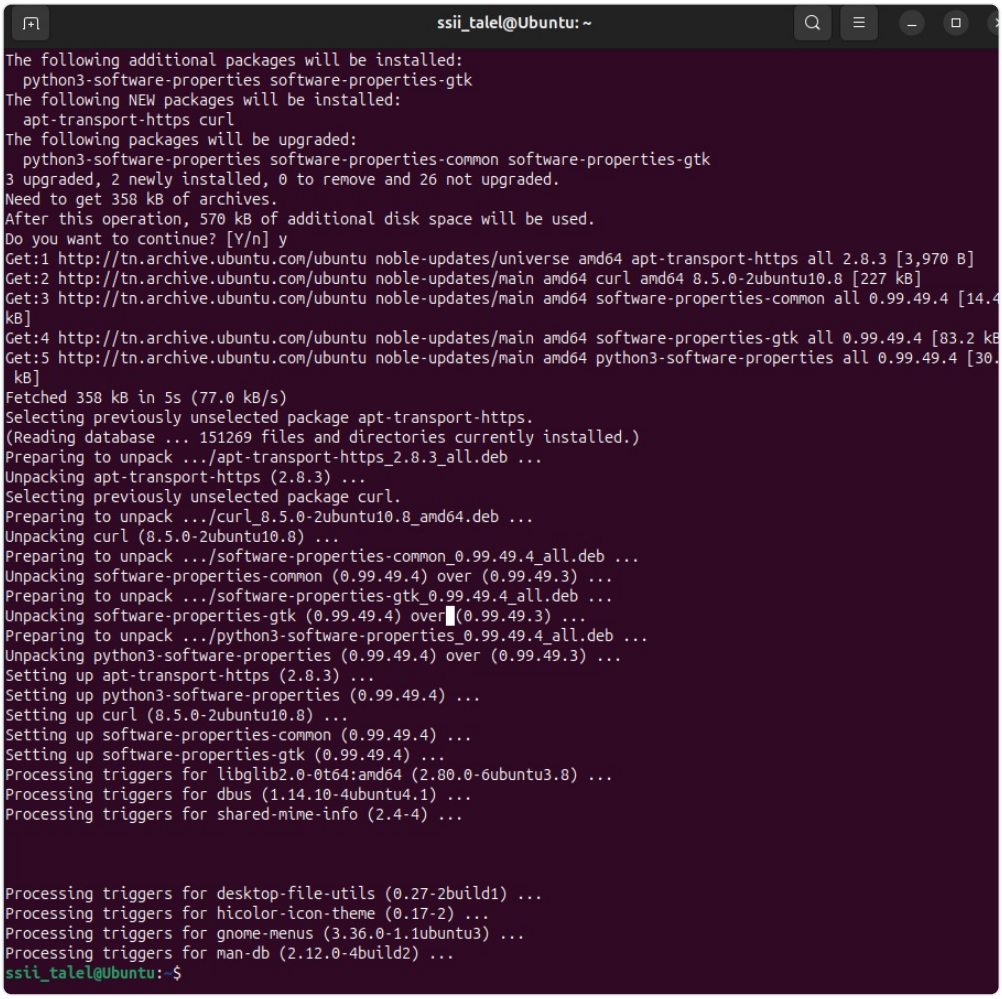


Figure 6 : Installation des dépendances nécessaires à Docker

3. Ajout de la clé GPG du dépôt Docker

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/docker-archive-keyring.gpg
```

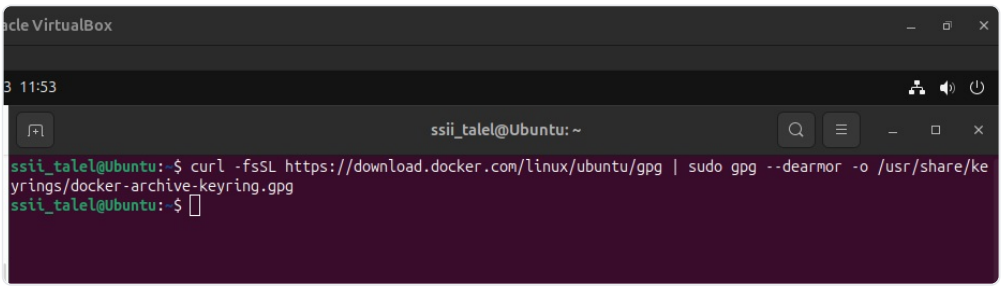


Figure 7 : Ajout de la clé GPG officielle du dépôt Docker

4. Configuration du dépôt stable Docker CE

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-
archive-keyring.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable"
| sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

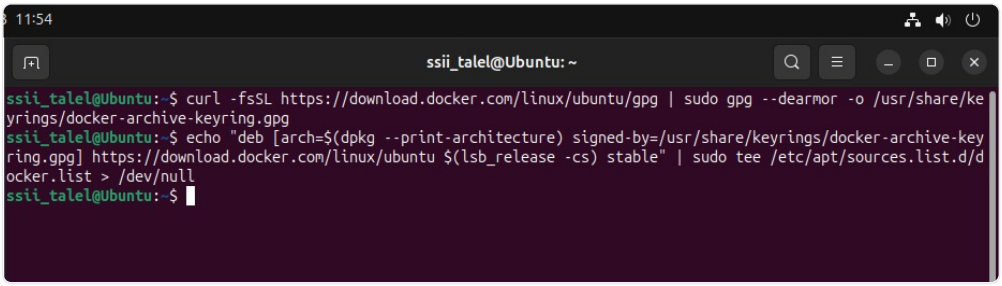


Figure 8 : Ajout du dépôt officiel Docker CE dans les sources APT

Partie 2 : Installer Docker CE

I. Installation de Docker CE et les paquets nécessaires

1. Mettre à jour l'index APT

```
sudo apt-get update
```

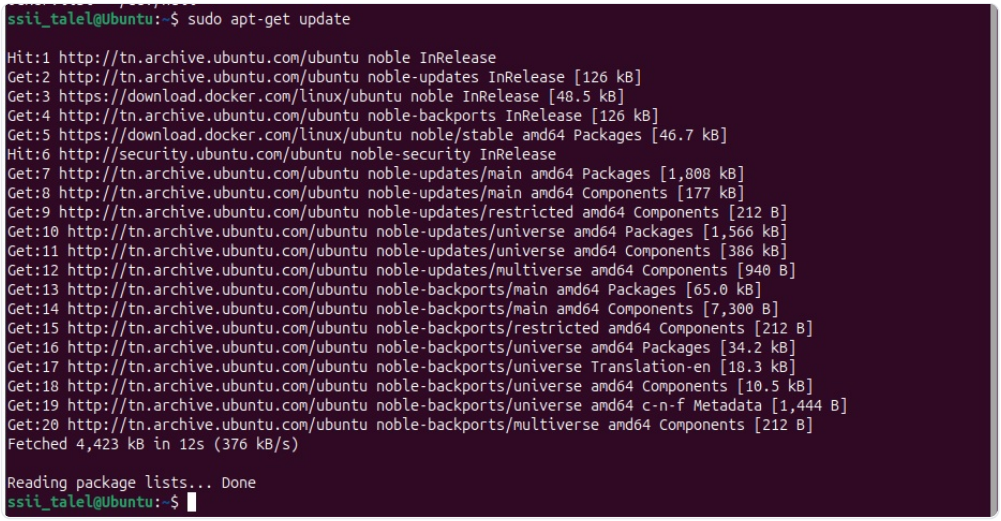


Figure 9 : Résultat de `sudo apt-get update`

---

2. Installer la dernière version de Docker Engine et containerd

```
sudo apt-get install docker-ce docker-ce-cli containerd.io
```

```
Selecting previously unselected package pigz.
Preparing to unpack .../03-pigz_2.8-1_amd64.deb ...
Unpacking pigz (2.8-1) ...
Selecting previously unselected package docker-buildx-plugin.
Preparing to unpack .../04-docker-buildx-plugin_0.31.1-1~ubuntu.24.04~noble_amd64.deb ...
Unpacking docker-buildx-plugin (0.31.1-1~ubuntu.24.04~noble) ...
Selecting previously unselected package docker-ce-rootless-extras.
Preparing to unpack .../05-docker-ce-rootless-extras_5%3a29.3.0-1~ubuntu.24.04~noble_amd64.deb ...
Unpacking docker-ce-rootless-extras (5:29.3.0-1~ubuntu.24.04~noble) ...
Selecting previously unselected package docker-compose-plugin.
Preparing to unpack .../06-docker-compose-plugin_5.1.0-1~ubuntu.24.04~noble_amd64.deb ...
Unpacking docker-compose-plugin (5.1.0-1~ubuntu.24.04~noble) ...
Selecting previously unselected package liberror-perl.
Preparing to unpack .../07-liberror-perl_0.17029-2_all.deb ...
Unpacking liberror-perl (0.17029-2) ...
Selecting previously unselected package git-man.
Preparing to unpack .../08-git-man_1%3a2.43.0-1ubuntu7.3_all.deb ...
Unpacking git-man (1:2.43.0-1ubuntu7.3) ...
Selecting previously unselected package git.
Preparing to unpack .../09-git_1%3a2.43.0-1ubuntu7.3_amd64.deb ...
Unpacking git (1:2.43.0-1ubuntu7.3) ...
Selecting previously unselected package libslirp0:amd64.
Preparing to unpack .../10-libslirp0_4.7.0-1ubuntu3_amd64.deb ...
Unpacking libslirp0:amd64 (4.7.0-1ubuntu3) ...
Selecting previously unselected package slirp4netns.
Preparing to unpack .../11-slirp4netns_1.2.1-1build2_amd64.deb ...
Unpacking slirp4netns (1.2.1-1build2) ...
Setting up liberror-perl (0.17029-2) ...
Setting up docker-buildx-plugin (0.31.1-1~ubuntu.24.04~noble) ...
Setting up containerd.io (2.2.2-1~ubuntu.24.04~noble) ...
Created symlink /etc/systemd/system/multi-user.target.wants/containerd.service → /usr/lib/systemd/system/container
d.service.
Setting up docker-compose-plugin (5.1.0-1~ubuntu.24.04~noble) ...
Setting up docker-ce-cli (5:29.3.0-1~ubuntu.24.04~noble) ...
Setting up libslirp0:amd64 (4.7.0-1ubuntu3) ...
Setting up pigz (2.8-1) ...
Setting up git-man (1:2.43.0-1ubuntu7.3) ...
Setting up docker-ce-rootless-extras (5:29.3.0-1~ubuntu.24.04~noble) ...
Setting up slirp4netns (1.2.1-1build2) ...
Setting up docker-ce (5:29.3.0-1~ubuntu.24.04~noble) ...
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service → /usr/lib/systemd/system/docker.servic
e.
Created symlink /etc/systemd/system/sockets.target.wants/docker.socket → /usr/lib/systemd/system/docker.socket.
Setting up git (1:2.43.0-1ubuntu7.3) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
ssii_talel@Ubuntu:~$
```

Figure 10 : Installation de Docker Engine et des composants associés

---

3. Vérifier la version de Docker installée

```
sudo docker --version
```

```
Setting up git (1:2.43.0-1ubuntu7.3) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
ssii_talel@Ubuntu:~$ sudo docker --version
Docker version 29.3.0, build 5927d80
ssii_talel@Ubuntu:~$
```

Figure 11 : Vérification de la version de Docker installée

III. Autoriser l'utilisateur à utiliser Docker sans sudo

Par défaut, seul `root` peut lancer les commandes Docker. Voici comment autoriser votre compte utilisateur.

I. Créer le groupe docker (s'il n'existe pas)

```
sudo groupadd docker
```

```
ssii_talel@Ubuntu: ~
ssii_talel@Ubuntu:~$ sudo groupadd docker
groupadd: group 'docker' already exists
ssii_talel@Ubuntu:~$
```

Figure 12 : Création du groupe `docker`

II. Ajouter l'utilisateur au groupe docker

```
sudo usermod -aG docker $USER
```

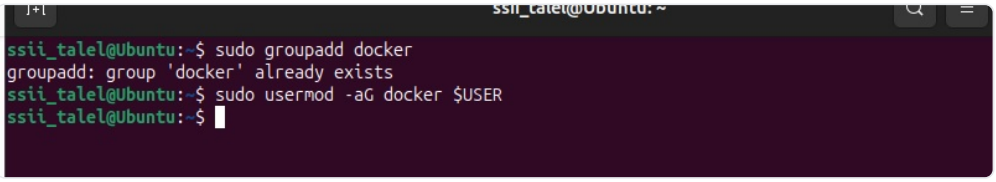


Figure 13 : Ajout de l'utilisateur courant au groupe `docker`

III. Redémarrer le système

```
reboot
```

IV. Vérifier que Docker fonctionne sans sudo

```
docker --version
```

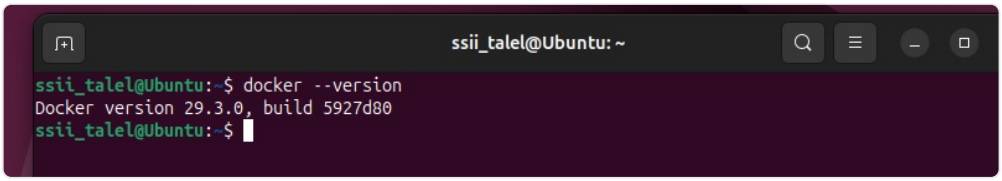


Figure 14 : Vérification que Docker fonctionne sans sudo (`docker --version` sans sudo)

V. Lancer le conteneur de test hello-world

```
docker run hello-world
```

Interprétez le résultat obtenu :

> ✏ **Réponse :** La commande `docker run hello-world` permet de vérifier que Docker fonctionne correctement. L'image `hello-world` est téléchargée (si elle n'est pas déjà présente), puis exécutée dans un conteneur. Le message affiché confirme que l'installation de Docker est réussie et que le moteur Docker est opérationnel. Le conteneur affiche un message de bienvenue puis s'arrête automatiquement.

```
ssii_talel@Ubuntu: ~
ssii_talel@Ubuntu:~$ docker --version
Docker version 29.3.0, build 5927d80
ssii_talel@Ubuntu:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
ea52d2000f90: Download complete
17eec7bbc9d7: Pull complete
Digest: sha256:85404b3c53951c3ff5d40de0972b1bb21fafa2e8daa235355baf44f33db9dbdd
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/

ssii_talel@Ubuntu:~$
```

Figure 15 : Message de succès après l'exécution du conteneur hello-world

Partie 3 : Manipulation d'images Docker

I. Lister les images Docker

```
docker image ls
```

ou bien :

```
docker images
```

```
ssii_talel@Ubuntu: ~
ssii_talel@Ubuntu:~$ docker image ls
                                                    i Info →  U  In Use
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest   85404b3c5395       25.9kB         9.52kB    U
ssii_talel@Ubuntu:~$ docker images
                                                    i Info →  U  In Use
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest   85404b3c5395       25.9kB         9.52kB    U
ssii_talel@Ubuntu:~$
```

Figure 16 : Résultat de `docker images` — liste des images locales

II. Supprimer une image Docker

1. Supprimer une image par son ID ou son nom

Avec l'ID de l'image :

```
docker rmi <IMAGE ID>
```

Avec le nom de l'image :

```
docker rmi <nom de l'image>
```

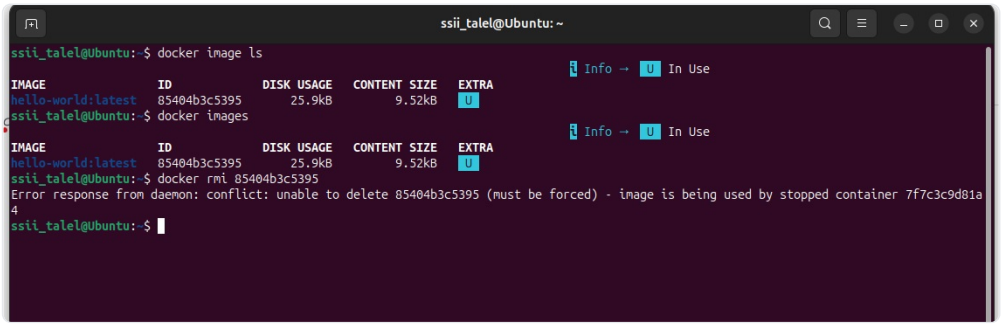


Figure 17 : Message d'erreur lors de la tentative de suppression d'une image Docker

---

2. Message d'erreur — Explication

Expliquez pourquoi vous obtenez un message d'erreur ?

> ✏ **Réponse :** Ce message d'erreur apparaît généralement lorsque l'image Docker que vous essayez de supprimer est utilisée par un ou plusieurs conteneurs (même arrêtés). Docker empêche la suppression d'une image tant qu'elle est référencée par un conteneur pour éviter les incohérences. Il faut d'abord supprimer ou détacher les conteneurs associés avant de pouvoir supprimer l'image.

---

3. Forcer la suppression de l'image

```
docker rmi -f hello-world
```

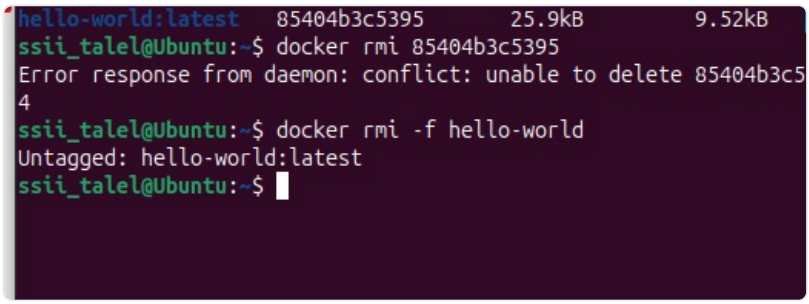


Figure 18 : Résultat de la suppression forcée d'une image Docker (`docker rmi -f hello-world`)

Partie 4 : Manipulation de conteneurs Docker

I. Créer un conteneur

1. Créer une instance d'une image (créer un conteneur)

```
docker run ubuntu:latest
```

```
ssii_talel@Ubuntu:~$ docker run ubuntu:latest
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
01d7766a2e4a: Downloading [==============================>                    ]  17.83MB/29.73MB
01d7766a2e4a: Downloading [==============================>                    ]  18.87MB/29.73MB
01d7766a2e4a: Pull complete

Digest: sha256:d1e2e92c075e5ca139d51a140fff46f84315c0fdce203eab2807c7e495eff4f9
Status: Downloaded newer image for ubuntu:latest
ssii_talel@Ubuntu:~$
ssii_talel@Ubuntu:~$
```

Figure 19 : Création d'un conteneur Ubuntu

---

2. Lancer un conteneur Ubuntu interactif

```
docker run -ti ubuntu:latest
```

Que remarquez-vous et pourquoi ?

> ✏ **Réponse :** *Le conteneur reste actif en mode interactif grâce à -ti. On peut travailler dans ce mini système Linux isolé.*

```
01d7766a2e4a: Pull complete

Digest: sha256:d1e2e92c075e5ca139d51a140fff46f84315c0fdce203eab2807c7e495eff4f9
Status: Downloaded newer image for ubuntu:latest
ssii_talel@Ubuntu:~$
ssii_talel@Ubuntu:~$ docker run -ti ubuntu:latest
root@51ef341eda4a:/#
```

Figure 20 : Lancement interactif d'un conteneur Ubuntu (`docker run -ti ubuntu:latest`)

---

3. Lancer la commande `ls` dans le conteneur

```
ls
```

```
01d7766a2e4a: Pull complete

Digest: sha256:d1e2e92c075e5ca139d51a140fff46f84315c0fdce203eab2807c7e495eff4f9
Status: Downloaded newer image for ubuntu:latest
ssii_talel@Ubuntu:~$
ssii_talel@Ubuntu:~$ docker run -ti ubuntu:latest
root@51ef341eda4a:/# ls
bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
root@51ef341eda4a:/#
```

Figure 21 : Résultat de la commande `ls` dans le conteneur

---

4. Lancer la commande `ip address`

```
ip address
```

```
01d7766a2e4a: Pull complete

Digest: sha256:d1e2e92c075e5ca139d51a140fff46f84315c0fdce203eab2807c7e495eff4f9
Status: Downloaded newer image for ubuntu:latest
ssii_talel@Ubuntu:~$
ssii_talel@Ubuntu:~$ docker run -ti ubuntu:latest
root@51ef341eda4a:/# ls
bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
root@51ef341eda4a:/# ip address
bash: ip: command not found
root@51ef341eda4a:/#
```

Figure 22 : Résultat de `ip address` dans le conteneur

---

5. Que remarquez-vous ?

> ✏ **Réponse :** *L'outil réseau (iproute2) n'est pas installé par défaut dans l'image Ubuntu minimale.*

---

6. Installer `iproute2` et obtenir l'adresse IP du conteneur

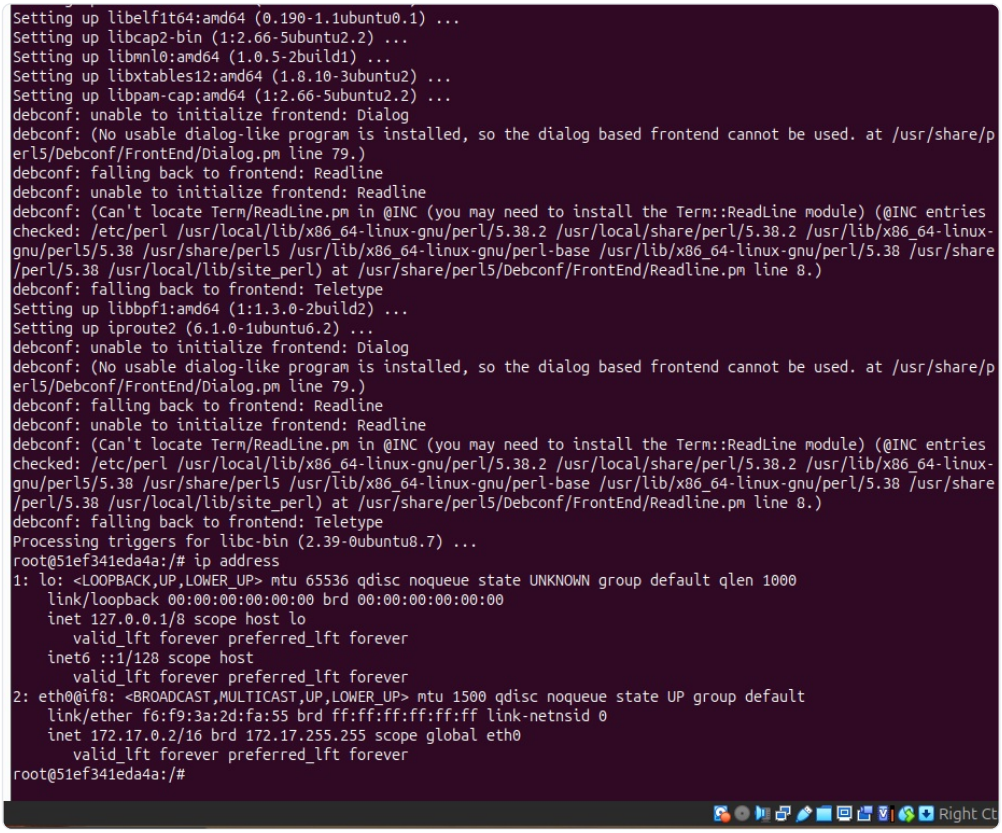
```
apt-get update && apt-get install iproute2
```

Puis relancer :

```
ip address
```

Donnez l'adresse IP de votre conteneur :

> ✏ **Réponse :** *eth0: inet 172.17.0.2/16*



```
Setting up libelf1t64:amd64 (0.190-1.1ubuntu0.1) ...
Setting up libcap2-bin (1:2.66-5ubuntu2.2) ...
Setting up libmnl0:amd64 (1.0.5-2build1) ...
Setting up libxtables12:amd64 (1.8.10-3ubuntu2) ...
Setting up libpam-cap:amd64 (1:2.66-5ubuntu2.2) ...
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog based frontend cannot be used. at /usr/share/p
erl5/Debconf/FrontEnd/Dialog.pm line 79.)
debconf: falling back to frontend: Readline
debconf: unable to initialize frontend: Readline
debconf: (Can't locate Term/ReadLine.pm in @INC (you may need to install the Term::ReadLine module) (@INC entries
checked: /etc/perl /usr/local/lib/x86_64-linux-gnu/perl/5.38.2 /usr/local/share/perl/5.38.2 /usr/lib/x86_64-linux-
gnu/perl5/5.38 /usr/share/perl5 /usr/lib/x86_64-linux-gnu/perl-base /usr/lib/x86_64-linux-gnu/perl/5.38 /usr/share
/perl/5.38 /usr/local/lib/site_perl) at /usr/share/perl5/Debconf/FrontEnd/Readline.pm line 8.)
debconf: falling back to frontend: Teletype
Setting up libbpf1:amd64 (1:1.3.0-2build2) ...
Setting up iproute2 (6.1.0-1ubuntu6.2) ...
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog based frontend cannot be used. at /usr/share/p
erl5/Debconf/FrontEnd/Dialog.pm line 79.)
debconf: falling back to frontend: Readline
debconf: unable to initialize frontend: Readline
debconf: (Can't locate Term/ReadLine.pm in @INC (you may need to install the Term::ReadLine module) (@INC entries
checked: /etc/perl /usr/local/lib/x86_64-linux-gnu/perl/5.38.2 /usr/local/share/perl/5.38.2 /usr/lib/x86_64-linux-
gnu/perl5/5.38 /usr/share/perl5 /usr/lib/x86_64-linux-gnu/perl-base /usr/lib/x86_64-linux-gnu/perl/5.38 /usr/share
/perl/5.38 /usr/local/lib/site_perl) at /usr/share/perl5/Debconf/FrontEnd/Readline.pm line 8.)
debconf: falling back to frontend: Teletype
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
root@51ef341eda4a:/# ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: eth0@if8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether f6:f9:3a:2d:fa:55 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 172.17.0.2/16 brd 172.17.255.255 scope global eth0
       valid_lft forever preferred_lft forever
root@51ef341eda4a:/#
```

Figure 23 : Adresse IP affichée pour le conteneur (ex. 172.17.0.2/16)

II. Utilité des conteneurs

1. Détruire le conteneur Ubuntu

> ⚠ **ATTENTION : Assurez-vous bien que vous êtes dans un conteneur avant de lancer cette commande !**

```
rm -rf /*
```

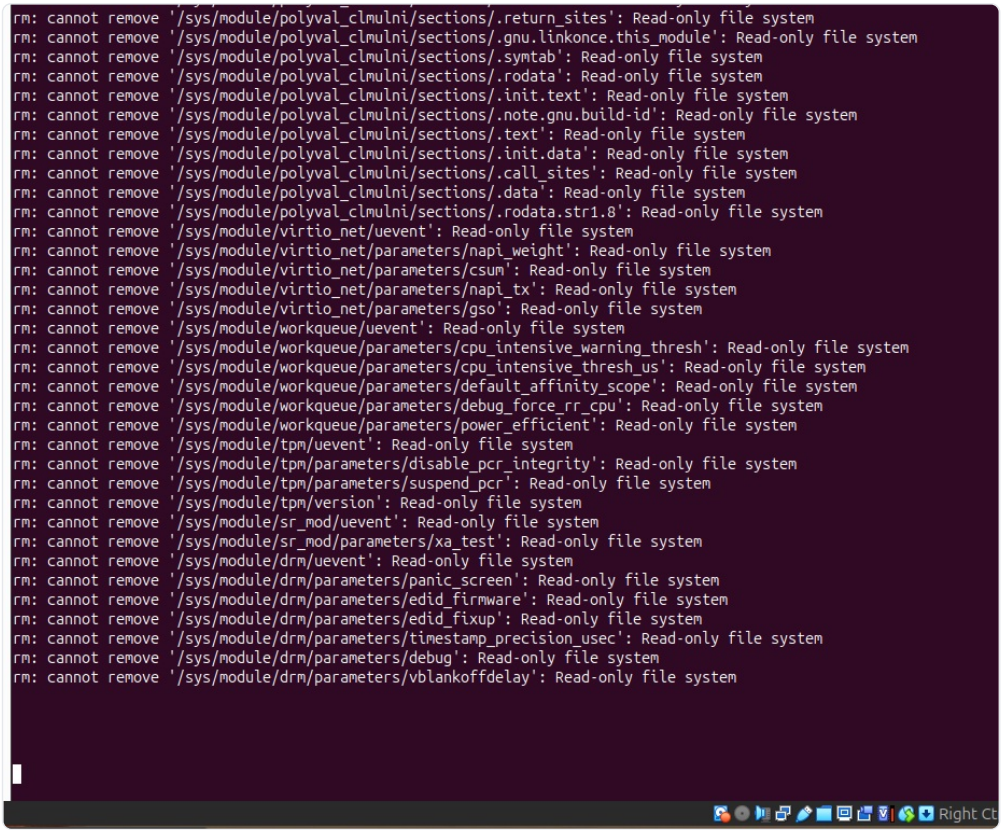


Figure 24 : Suppression des fichiers dans le conteneur (`rm -rf /*`) — exemple d'effet

---

2. Vérifier la destruction

```
ls
```

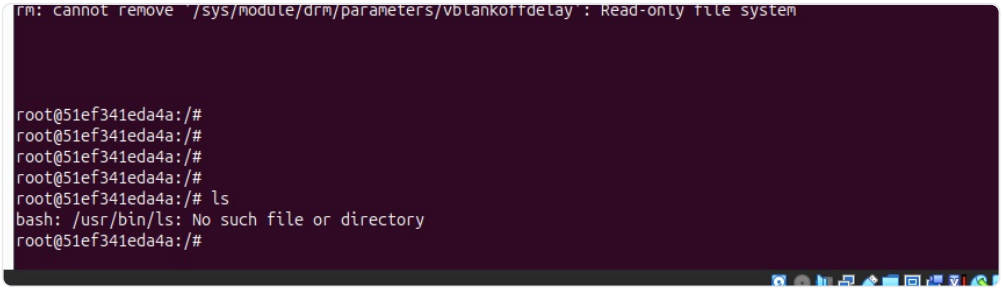


Figure 25 : Vérification de la destruction via `ls`

---

3. Quitter le conteneur et en créer un nouveau nommé `conteneur_ubuntu`

```
exit
```

```
docker run -ti --name conteneur_ubuntu ubuntu:latest
```

---

4. Relancer la commande `ls`

```
ls
```

```
root@f8c89dc26a07:/
ssii_talel@Ubuntu:~$ docker run -ti --name conteneur_ubuntu ubuntu:latest
root@f8c89dc26a07:/# ls
bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
root@f8c89dc26a07:/#
```

Figure 26 : Résultat de `ls` dans le nouveau conteneur `conteneur_ubuntu`

---

5. Que remarquez-vous ?

> ✏ **Réponse :** *Le nouveau conteneur est intact : les modifications apportées à l'ancien conteneur n'affectent pas le nouveau.*

---

6. Lancer `ip address` dans le nouveau conteneur — Résultat et explication

```
ip address
```

Qu'obtenez-vous et pourquoi ?

> ✏ **Réponse :** *On lance l'installation de iproute2 d'abord puis on exécute ip address. Chaque conteneur possède sa propre adresse IP et son réseau isolé.*

```
root@f8c89dc26a07:/# ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: eth0@if10: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 6a:35:5e:d1:bc:db brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 172.17.0.2/16 brd 172.17.255.255 scope global eth0
       valid_lft forever preferred_lft forever
root@f8c89dc26a07:/#
```

Figure 27 : Résultat de `ip address` dans le nouveau conteneur

---

7. Quitter le conteneur sans le détruire

```
Ctrl + P + Q
```

```
root@f8c89dc26a07:/# clearssii_talel@Ubuntu:~$
```

Figure 28 : Illustration de la combinaison `Ctrl+P+Q` pour détacher du conteneur

---

III. Afficher la liste des conteneurs

```
docker container ls
```

ou :

```
docker ps
```

```
root@f8c89dc26a07:/# clearssii_talel@Ubuntu:~$ docker container ls
CONTAINER ID   IMAGE           COMMAND       CREATED          STATUS          PORTS     NAMES
f8c89dc26a07   ubuntu:latest   "/bin/bash"   13 minutes ago   Up 13 minutes             conteneur_ubuntu
ssii_talel@Ubuntu:~$
```

Figure 29 : Résultat de `docker container ls` / `docker ps`

IV. Exécuter un shell dans le conteneur

```
docker exec -ti conteneur_ubuntu bash
```

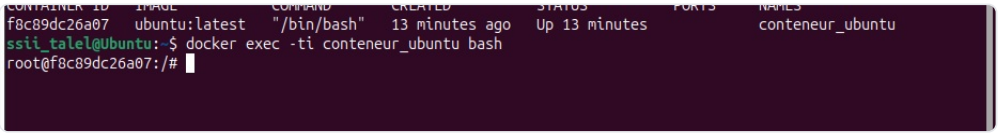


Figure 30 : Exécution d'un shell dans le conteneur (`docker exec -ti conteneur_ubuntu bash`)

V. Transformer un conteneur en image

1. Installer `iproute2` (net-tools)

```
apt-get update && apt-get install iproute2
```

2. Créer un fichier de test

```
echo "ceci est un fichier qui contient des donnes de test" > test.txt && cat test.txt
```

Figure 31 : Création et affichage du fichier `test.txt` dans le conteneur

3. Quitter le conteneur sans le détruire

```
exit
```

4. Transformer le conteneur en image nommée `image_m1ssii`

```
docker commit <CONTAINER NAME or ID> image_m1ssii
```

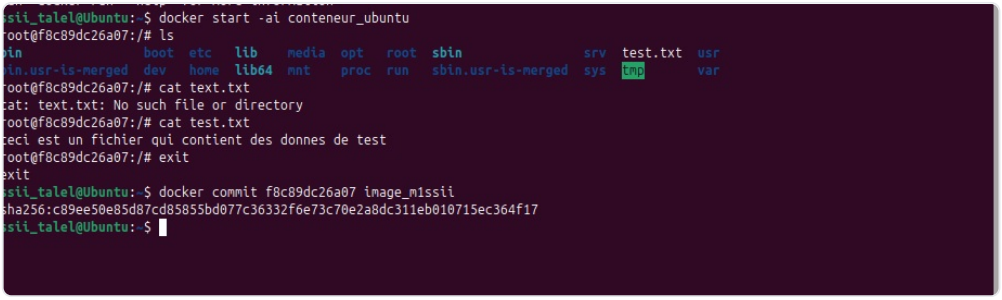


Figure 32 : Résultat de la commande `docker commit` — création de `image_m1ssii`

5. Vérifier la nouvelle image

```
docker images
```

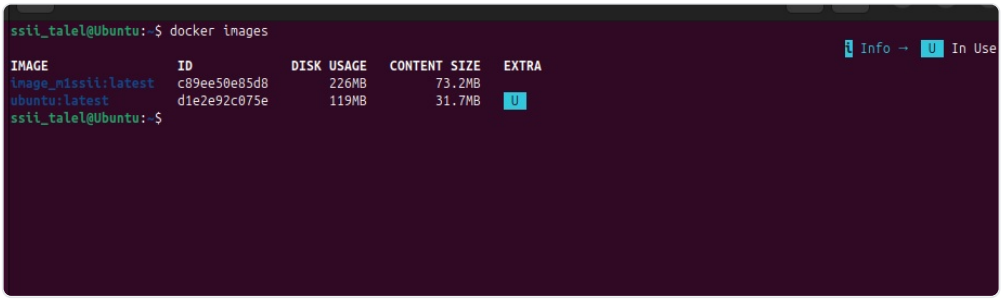


Figure 33 : Affichage de `docker images` montrant `image_m1ssii`

---

6. Relancer un nouveau conteneur nommé `conteneur_m1ssii` basé sur la nouvelle image

```
docker run -ti --name conteneur_m1ssii image_m1ssii
```

---

7. Vérifier si les données ont bien été stockées

Vérifiez si le fichier `test.txt` est présent dans ce nouveau conteneur :

> ✏ **Réponse :** *Les données ont été conservées dans la nouvelle image et sont présentes dans le conteneur créé à partir de cette image.*

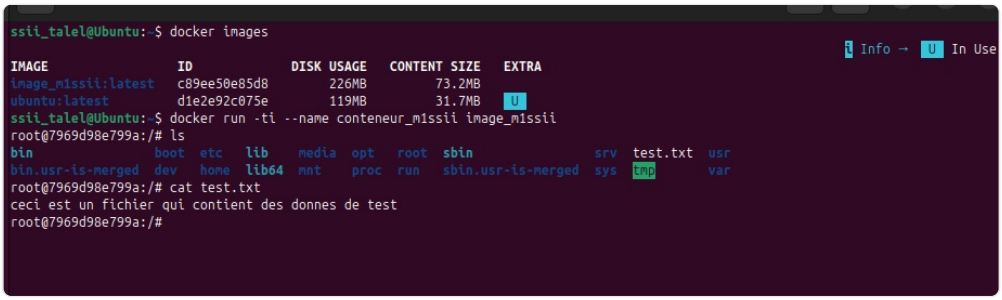


Figure 34 : Vérification que `test.txt` est présent dans le conteneur créé à partir de `image_m1ssii`

VI. Supprimer un conteneur

```
docker rm <CONTAINER NAME ou ID>
```

Vérifiez que le conteneur a bien été supprimé :

```
docker ps -a
```

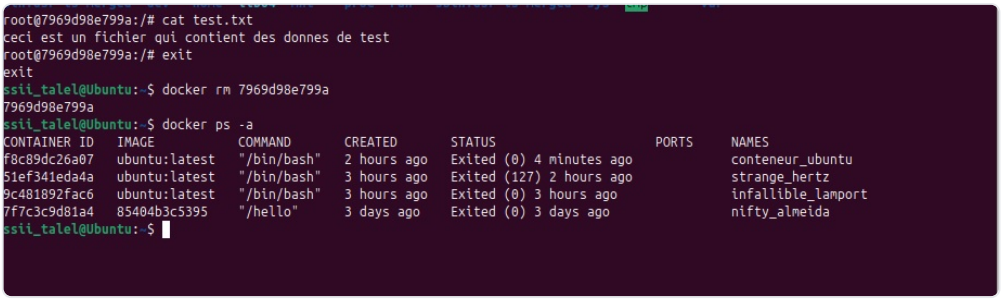


Figure 35 : Affichage de `docker ps -a` confirmant la suppression du conteneur

---

Bonnes pratiques de sécurité Docker/VM

Voici quelques recommandations à appliquer lors de l'utilisation de Docker sur une machine virtuelle :

1. **Ne pas utiliser root pour tout**
    - Créez un utilisateur dédié pour l'administration courante.
    - N'utilisez `sudo` que pour les opérations nécessaires.
2. **Limiter les droits Docker**
    - Évitez d'ajouter tous les utilisateurs au groupe `docker` (ce groupe donne les droits root sur la machine).
    - Privilégiez l'utilisation de `sudo docker ...` pour les opérations sensibles.
3. **Sécuriser les conteneurs**
    - Ne jamais lancer un conteneur avec `--privileged` sauf nécessité absolue.
    - Utilisez l'option `--read-only` pour les conteneurs qui n'ont pas besoin d'écrire sur le système de fichiers.
    - Montez explicitement les volumes nécessaires, en lecture seule si possible (`-v /host/path:/container/path:ro`).
4. **Réseau**
    - N'exposez que les ports nécessaires (`-p 8080:80` et pas `-p 0.0.0.0:80:80` si ce n'est pas utile).
    - Utilisez des réseaux bridge personnalisés pour isoler les conteneurs.
5. **Images**
    - Utilisez des images officielles ou vérifiées.
    - Mettez à jour régulièrement les images et supprimez celles qui sont obsolètes.
6. **Suppression de conteneurs/images**
    - Ne jamais exécuter de commandes destructives (`docker rm -f $(docker ps -aq)` ou `docker rmi -f $(docker images -q)`) sans vérification.
    - Toujours vérifier l'ID ou le nom du conteneur/image avant suppression.
7. **Logs et audit**
    - Activez la journalisation des actions Docker.
    - Surveillez les accès et les modifications sur la VM.
8. **Sauvegarde**
    - Sauvegardez régulièrement les volumes de données importants.
9. **Sécurité système**
    - Gardez la VM et Docker à jour (patchs de sécurité).
    - Désactivez SSH root direct, utilisez des clés SSH, etc.

Fin du TP 1 — Initiation à Docker